Empirically Comparing the Test Suite Reduction Techniques in Continuous Integration Process

How Test Suite Reduction Impacts Fault Localization

Jutarporn Intasara
Department of Computer Science and Information Engineering, National Chiayi University, Taiwan
s1040464@mail.ncyu.edu.tw

Chu-Ti Lin*
Department of Computer Science and Information Engineering, National Chiayi University, Taiwan
chutilin@mail.ncyu.edu.tw

ABSTRACT

In a continuous integration (CI) environment, software developers can frequently integrate the new or modified code to codebase and ensure the correctness of the modification by performing regression testing (RT). RT generally takes long time but the time period allocated for it is limited in practice. A CI environment can adopt test suite reduction (TSR) to improve RT and accelerate debugging by performing spectrum-based fault localization (SBFL). Yet, different TSR techniques will achieve different program test spectra and SBFL will operate based on these test spectra. Thus, the selection of TSR techniques has influence on both the effectiveness of the fault localization (FL) and the length of a CI period. If developers want to improve the CI by including TSR and SBFL, they have to choose a suitable TSR technique to maximize the FL effectiveness and minimize the CI period length. In this paper, we consider a TSR technique as the combination of the TSR strategy, the metric to evaluate test cases, and the coverage granularity level. We will analyze the impact caused by these three factors. Our study covers three TSR strategies, four metrics to evaluate test cases, and three coverage granularity levels, thus resulting in 36 TSR techniques. We investigate how each of the 36 TSR techniques affects the FL effectiveness and the CI period length by conducting the experiment based on seven subject programs that have been frequently adopted in the relevant studies. The experiment results indicate that performing TSR at branch coverage is recommended for optimizing the FL effectiveness while the HGS algorithm is suggested if the CI period length is developers' main concern.

CCS CONCEPTS

• **Software and its engineering**; • **Software creation and management**; • **Software verification and validation**; • **Software defect analysis**; • **Software testing and debugging**;

*Corresponding author.

ESCC '21, April 16–18, 2021, Belgrade, Serbia

ACM ISBN 978-1-4503-8749-1/21/04…$15.00
https://doi.org/10.1145/3478301.3478304

KEYWORDS

Software Testing, Debugging, Continuous Integration, Test Suite Reduction, Fault Localization

ACM Reference Format:
Jutarporn Intasara and Chu-Ti Lin. 2021. Empirically Comparing the Test Suite Reduction Techniques in Continuous Integration Process: How Test Suite Reduction Impacts Fault Localization. In *The 2nd European Symposium on Computer and Communications (ESCC '21), April 16–18, 2021, Belgrade, Serbia.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3478301.3478304

1 INTRODUCTION

Software regression testing (RT) is an activity to ensure the correctness of the modified parts by running the test cases of the modified parts together with those that have been executed for the previous versions [1]. Thus, performing RT generally takes a lot of time. According to [2], test suite reduction (TSR) is one of the popular strategies to improve RT that produces a representative set (RS) by removing the redundant test cases from the original test suite (OTS). On the other hand, if faults are revealed by the testing, software debugging should be taken to avoid the same failure during operation. Software debugging tries to pinpoint the root cause of the failures, namely fault localization (FL), and design and implement the code repair. In decades, many automatic FL techniques have been proposed to accelerate the debugging process and make software more reliable and maintainable [3]. Among these techniques, spectrum-based fault localization (SBFL) has received considerable attention since it has achieved significant result with low overheads in comparison to the others [4].

Continuous integration (CI) [5, 6] can help developers simultaneously modify or integrate the new code to the code repository. A CI environment will become desirable if TSR and SBFL are both included to improve RT and FL. More specifically, a TSR technique can be utilized to reduce the test suite size and a SBFL technique can then be adopted to speed up the fault fix after RT. That is, TSR and SBFL can be incorporated into the CI environment for shortening the time taken for testing and debugging (i.e., the length of a CI period will be reduced). However, different TSR techniques will produce different RS and then result in different program test spectra (i.e., code coverage and test results). In addition, a SBFL technique will use a fault locator (i.e., the formula taken to evaluate each code statement's probability of containing a fault) according to the program test spectra to predict the locations of faults. As a result, both the FL effectiveness and the CI period length are influenced by the selection of TSR techniques.

Some researchers discussed the relationship between RT techniques and FL. Jiang et al. [6] studied the influence on FL in a CI process caused by the three different factors of test case prioritization (TCP) (i.e., the strategy, the coverage granularity, and the time cost). Different from Jiang et al.'s work, we focus on another RT technique, i.e., "TSR", to study how it impacts on FL. On the other hand, Yu et al.'s work [7] investigated the effect of TSR on the effectiveness of SBFL and the Greedy algorithm is acted as the TSR strategy for their empirical study. According to their empirical results, reducing the test suite size with the Greedy algorithm based on statement coverage for minimizing the testing cost is suggested. Yet, they also proposed a new TSR metric, called vector-based, and suggested it to replace the statement-based metric for maximizing FL effectiveness. Because Yu et al.'s work adopted only one TSR technique, our study further investigates how the other algorithms (e.g., GRE and HGS) at the different coverage granularity levels (i.e., branch and function) impact the FL effectiveness and the CI period length. Moreover, their work only evaluated the importance of test cases according to the code coverage, software developers still cannot understand the impact caused by adopting another way to evaluate test cases during TSR. Thus, we further consider the other two ways to evaluate test cases in our study.

The remainder of this paper is organized as follows. Section 2 will briefly review TSR and SBFL. Section 3 will describe our research questions, describe the criteria used to evaluate the FL effectiveness and the CI period length in our empirical study, and the experimental setup. Section 4 will answer the research questions and highlight our findings from the experiment. Finally, Section 5 will furnish some concluding remarks.

2 BACKGROUND

2.1 Code coverage granularities

TSR removes the redundant test cases from the OTS and tries to produce the RS that satisfies a specific criterion [2, 8]. In the literature, many TSR techniques (e.g., the techniques described in [9, 10]) regard the test requirement as a code statement, a branch, or a function. For example, if the coverage granularity level adopted by developers to perform TSR is statement, the test cases in the RS should cover all of the code statements that are covered by those in the OTS. Because performing TSR at different coverage granularities will produce different RS and then affect the consequent SBFL, we consider coverage granularity as an important factor for TSR. In addition to coverage granularity level, the metric to evaluate test cases (hereafter called reduction metric for simplicity) and the TSR strategy are the other two important factors to design a TSR technique. Thus, our work defines a TSR technique as a combination of a TSR strategy, a reduction metric, and a coverage granularity. Section 2.2 and 2.3 will review several TSR strategies and reduction metrics, respectively.

2.2 Review of TSR strategies

In the literature, a lot of TSR strategies have been proposed. In this subsection, we will review three well-known TSR strategies that will be adopted in our empirical study.

AG: In the TSR process, the Additional Greedy algorithm (called AG hereafter) [9] repeatedly chooses a test case that can satisfy the most unsatisfied test requirements and includes it in the RS until all of the test requirements are satisfied.

HGS: This heuristic algorithm is called HGS due to the abbreviations of the authors' names (i.e., Harrold, Gupta, and Soffa) [11]. Let S_j (for $j = 1, 2, 3, \ldots, n$) represent the subset of unselected test cases in the OTS that can satisfy the j-th test requirement. The selection process starts from the S_j with the smallest cardinality, moves the test cases in S_j that belong to the most subsets of the same cardinality to the RS, marks all of the test requirements that are satisfied by the selected test case, and removes those test cases from all S_j. If a tie occurs during selecting test cases in S_j, it will recursively consider the number of unsatisfied test requirements that are satisfied by the candidates and choose the test case that can satisfy the most unsatisfied test requirements. The aforementioned steps will be repeated until all of the test requirements are satisfied.

GRE: It is called GRE [12] because it includes three strategies (i.e., the AG strategy, the 1-to-1 redundancy strategy, and the essentials strategy). More specifically, a test case is called essential if it can satisfy the test requirement that cannot be satisfied by the others. The essential test cases will be included in the RS first. Additionally, the 1-to-1 strategy will remove a test case from the OTS if all of the test requirements satisfied by this test can be satisfied by any test case in the RS. These two strategies will be repeatedly applied together with the AG until all of the test requirements are satisfied.

2.3 Reduction metrics

During the TSR process, the importance of test cases will be evaluated according to a specific metric. The four reduction metrics adopted by Lin et al. in [9] are described as follows.

Cov: Coverage metric (hereafter called Cov) measures the test cases' importance according to the number of satisfied test requirements. The TSR techniques that adopt this metric assume that the test case covering more test requirements should have a higher chance to reveal faults.

Ratio: In addition to the number of satisfied test requirements, Smith and Kapfhammer [13] also took into account the time taken to run test cases. Thus, they defined a cost-aware metric called Ratio metric (called Ratio hereafter) as the coverage increase per unit of cost consumption and used it to evaluate test cases.

Irre: This metric (hereafter called Irre) was proposed in [9] based on the concept of irreplaceability. More specifically, test case t is supposed to have high irreplaceability if it is not easy to find another test case to satisfy the test requirements that can be satisfied by t. Based on this concept, Lin et al. [9] proposed this cost-aware metric to evaluate test cases' contribution on increasing code coverage.

EIrre: Lin et al. [9] also considered the essentials strategy and posited that the essential test cases should be given the highest priority. They further proposed an enhancement of Irre called EIrreplaceability (hereafter called EIrre) by giving the essential test cases the highest irreplaceability.

2.4 SBFL techniques

SBFL [14] is an approach to identify the location of the bugs for the debugging process. This approach leverages the program test spectra to evaluate the probability of each code statement being faulty, called the suspiciousness. All of the code statements will then

Table 1: Description of the adopted SBFL techniques

SBFL Technique	Fault Locator
Jaccard [16]	$J\_sus(s) = \frac{FE(s)}{FT+PE(s)}$
Ochiai [16]	$O\_sus(s) = \frac{FE(s)}{\sqrt{FT \times (FE(s)+PE(s))}}$
SBI [17]	$S\_sus(s) = \frac{FE(s)}{PE(s)+FE(s)}$
Tarantula [18]	$T\_sus(s) = \frac{FE(s)/FT}{(FE(s)/FT)+(PE(s)/PT)}$, $Conf(s) = max(\frac{FE(s)}{FT}, \frac{PE(s)}{PT})$

be ranked according to their suspiciousness and the code statements with higher suspiciousness will be examined early. Table 1 shows four SBFL techniques that have received considerable attention in the literature [15]. Thus, our empirical study will include these four techniques.

To facilitate the description of these techniques, we defined the following notations for program test spectra: *FT* represents the number of failed test cases, *PT* the number of passed test cases, *FE*(*s*) the number of failed test cases which executed the statement *s*, and *PE*(*s*) the number of passed test cases which executed *s*.

3 EMPIRICAL STUDY

3.1 Research questions

Three research questions will be discussed in our empirical study to help software developers determine the suitable TSR techniques in the CI environment from different standpoints.

RQ1: Which TSR techniques are suggested for optimizing the FL effectiveness?

RQ2: Which TSR techniques are suggested for reducing the length of a CI period?

RQ3: Which TSR techniques are suggested when considering both objectives?

3.2 Evaluation metrics

In this study, we evaluate the FL effectiveness and the CI period length for each of the 36 TSR techniques by using the *EXAM* score and the T_{CI} value, respectively. The *EXAM* score [3, 15] represents the percentage of statements in a faulty program that have to be examined until the first faulty statement is located. The lower *EXAM* score means higher effectiveness to locate faults. Let s_{chk} be the number of statements that have been examined until locating the first faulty statement and *LOC* the total number of statements in the faulty program. The *EXAM* score can be defined as follows.

$$EXAM = (s_{chk}/LOC) \times 100\%, \quad (1)$$

On the other hand, recall that a CI process can be automated after code commitment. The time taken to perform RT (i.e., the time periods taken to produce the RS (called T_{RTS}) and run the tests in the RS (called T_{RUN})) and perform FL (called T_{FL}) are measured and denoted by T_{CI}. The lower T_{CI} is; the shorter length of a CI period is. The T_{CI} value for each of the 36 TSR techniques can be defined as follows.

$$T_{CI} = T_{RTS} + T_{RUN} + T_{FL} \quad (2)$$

3.3 Experiment setup

We conducted the experiment by using the seven programs in the Siemens suite with 98 faulty versions. These programs are of small size and frequently used in prior studies [3, 18]. These subject programs together with their test pools are available in the Software-artifact Infrastructure Repository (SIR) [19].

Before conducting the experiment, we collected the coverage information at three different granularity levels (including branch, function, and statement) and the execution cost of the test cases in the Siemens suite. This process was taken on Windows 7 machine with 3.20 GHz Intel(R) Core(TM) i5 CPU and 16 GB of main memory modules. More specifically, for each faulty version of each subject, we ran each of the test cases in the test pool and gathered its coverage information by using the profiling tool gcov (version 6.4.0) [20]. Considering the execution cost, we repeatedly ran each test case three times, recorded the execution costs, and took the average across them. We repeatedly conducted our experiment for each of the 98 faulty versions 1,000 times by the following these three steps below.

Step 1: *Generate the OTS:* Randomly select v test cases from the test pool and include them to the OTS where $1 \leq v \leq LOC \times 0.5$. If there exist unsatisfied test requirements according to the three coverage criteria, we will repeatedly and randomly choose and include one more test case that can cover at least one of the unsatisfied test requirements until all of the test requirements are satisfied. If the OTS does not contain any failed test cases, we will further randomly choose v_f failed test cases from the test pool, and include them in the OTS, where $1 \leq v_f \leq n_f$ and n_f represents the total number of failed test cases in the test pool.

Step 2: *Perform TSR and run the RS:* According to the definition of TSR technique in Section 2.1, 3×4×3=36 different TSR techniques will compared in our study. We will perform each of the 36 TSR techniques to obtain a RS, run the test cases in each RS, and obtain the test spectra.

Step 3: *Perform SBFL:* Perform the four SBFL based on the results of Step 2 (i.e. test spectra), suggest the ranking of code statements to be examined during debugging, and compute the *EXAM* scores and the T_{CI} values.

4 RESULTS AND ANALYSES

To answer RQs 1 through 3, we repeated the experiment 1,000 times, obtained the results (i.e., *EXAM* scores and T_{CI} values), and took average across the 1,000 runs. We will evaluate the FL effectiveness and the CI period length of the 36 TSR techniques according to their

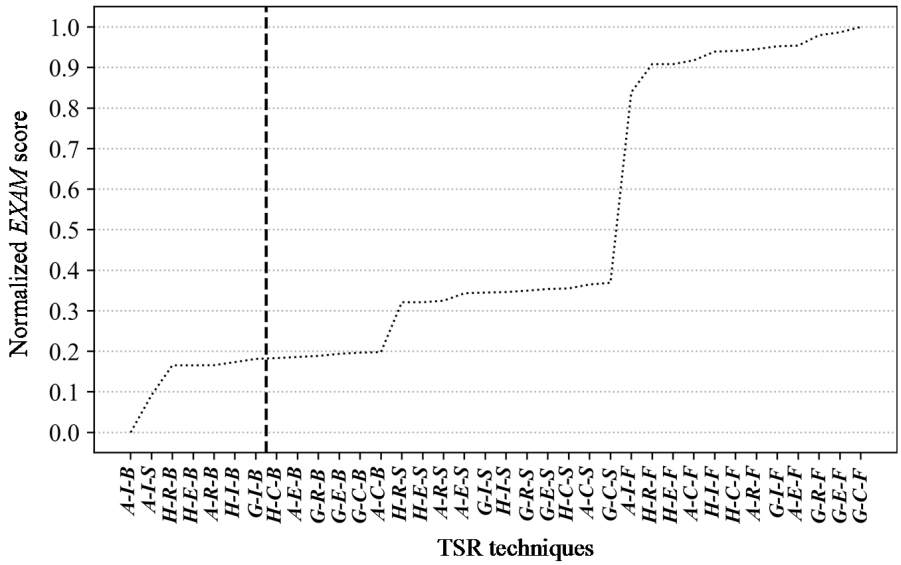

Figure 1: The comparison in FL effectiveness.

normalized *EXAM* scores and normalized T_{CI} values. More specifically, because the ranges of different measures are not consistent, we will normalize the obtained results so that the minimum will be 0 while the maximum will be 1. When visualizing the experimental results, we will also show all of the 36 TSR techniques in an ascending order according to their normalized values. We also will focus on the best 20% TSR techniques among the 36 ones (i.e., the top seven TSR techniques at the left-hand side of the vertical dashed line). We use *X-Y-Z* to represent a TSR technique, where *X*, *Y*, and *Z* represent the abbreviations of the TSR strategy, the reduction metric, and the coverage granularity level. The abbreviations of the three TSR strategies (i.e., AG, GRE, and HGS) are given by *A*, *G*, and *H* while the four reduction metrics (i.e., Cov, Ratio, Irre, and EIrre) are called *C*, *R*, *I*, and *E* for simplicity. Additionally, the three coverage granularities (namely branch, function, and statement) are denoted by *B*, *F*, and *S*. Thus, *A-C-B* represents a TSR technique that adopts Cov to evaluate the importance of a test case at branch coverage and then utilizes the AG algorithm to reduce test suites.

4.1 Answering to RQ1

RQ1 aims to determine the seven TSR techniques that achieve the best FL effectiveness (i.e., the seven TSR techniques with the lowest normalized *EXAM* scores). For each of the 36 TSR techniques, we evaluated its FL effectiveness by taking the average across 98×1,000×(1×1×1)×4=392,000 *EXAM* scores and the results were illustrated in Figure 1. In this figure, the *y*-axis represents all of the 36 TSR techniques' normalized *EXAM* scores while the *x*-axis represents these TSR techniques that were reordered according to their normalized *EXAM* scores. Considering the seven best TSR techniques in Figure 1, most of them reduce test suites at branch coverage, except for *A-I-S*. None of these seven TSR techniques adopt Cov to evaluate test cases' importance while four out of them adopt Irre. Please notice that if adopting the AG algorithm and utilizing Irre to evaluate the test cases' importance, performing TSR at both branch and statement coverage provides the best FL effectiveness. According to the aforementioned descriptions, we found that the coverage granularity level should have the most significant impact on FL effectiveness. Considering the other two factors, the reduction metric should have more impact on FL effectiveness than the TSR strategy.

Overall, if software developers decide to include TSR in the CI environment and their main consideration is FL effectiveness, performing TSR at branch coverage is recommended. Additionally, evaluating the importance of test case by Irre and adopting the AG algorithm may help them further improve FL effectiveness.

4.2 Answering to RQ2

Figure 2 compares the CI period length of the 36 TSR techniques in terms of T_{CI} value. Similar to the reply of RQ1, the T_{CI} value of each TSR technique is obtained by taking the average across 98×1,000×(1×1×1)×4= 392,000 T_{CI} values. In Figure 2, the *y*-axis indicates the normalized T_{CI} values of the 36 TSR techniques while the *x*-axis indicates these TSR techniques that were reordered according to their normalized T_{CI} values. As seen from the figure, all of the seven best TSR techniques adopt the HGS algorithm, thus indicating that the HGS algorithm should be the best TSR strategy for reducing the length of the CI period. Additionally, we found that the best four TSR techniques are performed at function coverage and the other three at branch coverage. It is also noted that all of the four reduction metrics were utilized in the seven best TSR

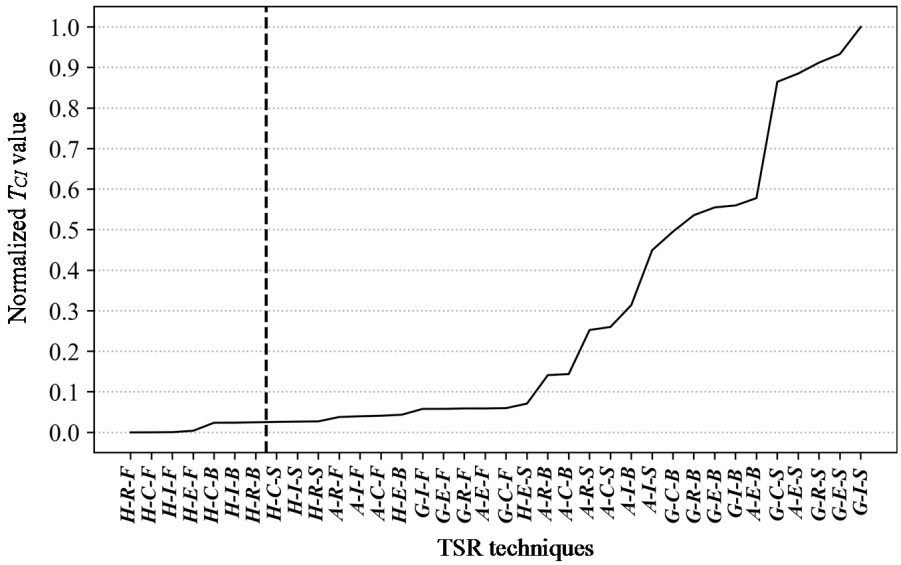

Figure 2: The comparison in the length of CI period.

techniques. Thus, we posit that the CI period length is sensitive to the selection of TSR strategy and coverage granularity. It is worth noting that TSR strategy almost dominates the ranking.

On the whole, if TSR is adopted the CI environment and controlling the CI period length is the most important concern for software developers, using the HGS algorithm is suggested. Additionally, adopting the HGS algorithm to reduce test suites based on function coverage may further reduce the CI period length.

4.3 Answering to RQ3

For RQ3, we consider both of the FL effectiveness and the CI period length in the CI environment. In Figure 3, the y-axis indicates the summation of the normalized *EXAM* score and the normalized T_{CI} value while the x-axis indicates these 36 TSR techniques that were reordered according to the summations. As seen from the figure, all of the seven best TSR techniques reduce the test suites at branch coverage. We also found that among these seven techniques, the best four adopt the HGS algorithm while the others use the AG algorithm. Although we can also see that the ranking for the best four TSR techniques in terms of reduction metric is Ratio, Irre, Cov, and EIrre, the selection of reduction metric has less impact in comparison with TSR strategy and coverage granularity.

Overall, if the FL effectiveness and the CI period length in the CI environment are both software developers' considerations, performing TSR at branch coverage is suggested. In addition, if software developers can further adopt the HGS algorithm, the results should be more desirable.

5 CONCLUSIONS AND FUTURE WORK

TSR and FL are two approaches to improve the software development process in a CI environment. Various TSR techniques and SBFL techniques result in a lot of possible configurations for the CI. In this paper, we discussed the influence on the FL effectiveness and the CI period length caused by 36 TSR techniques. We also conducted the experiments on seven subject programs that were frequently used in the relevant studies and drew some findings from the best 20% TSR techniques that were compared in this study. From the replies of RQs 1 through 3, we summarize our findings as follows: (1) if the FL effectiveness is the software developers' main concern, performing TSR at branch coverage is suggested; (2) the HGS algorithm is recommended to reduce the size of test suite when the software developers focus on the CI period length; (3) if both FL effectiveness and CI period length are considered, software developers should pay attention to the TSR strategy and the coverage granularity level. More specifically, performing the HGS algorithm at branch coverage is recommended for addressing this multi-objective criterion. This paper aims to develop a guidance for software developers to choose a suitable TSR technique during designing a CI environment in which SBFL is also considered. Yet, only seven subjects were analyzed in the current work. In future work, we plan to perform additional experiments on more large-scale programs to justify our findings.

ACKNOWLEDGMENTS

This work was supported by Ministry of Science and Technology Taiwan, under Grants MOST 108-2628-E-415-001-MY2.

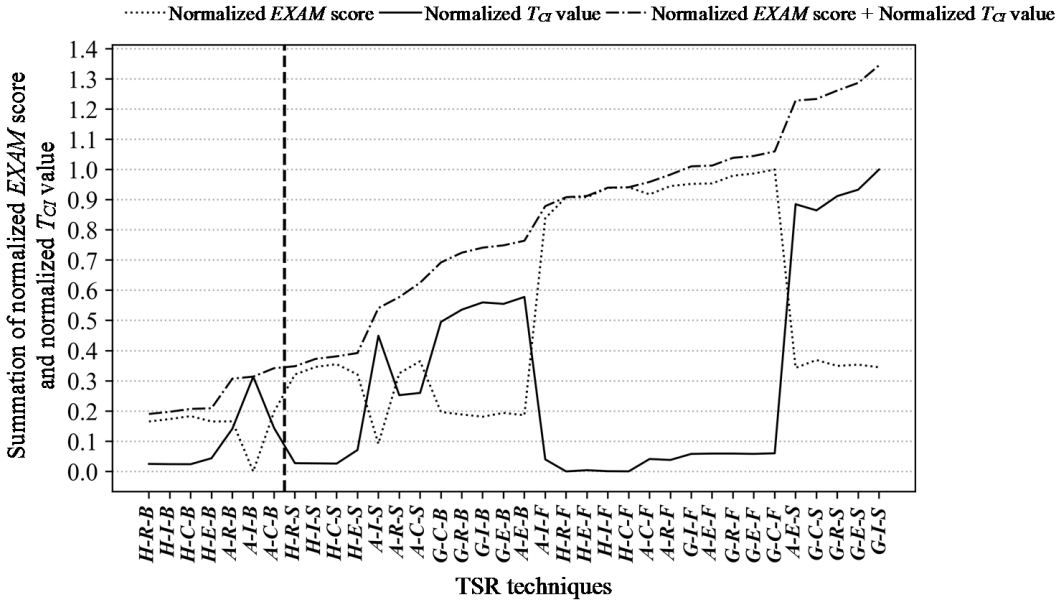

Figure 3: The comparison in the summation of FL effectiveness and the length of CI period.

REFERENCES

[1] Mauro Pezzè and Michal Young. 2008. Software Testing and Analysis: Process, Principles, and Techniques. John Wiley & Sons. USA.

[2] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: A survey. Software Testing, Verification and Reliability 22, 2 (March 2012), 67-120. https://doi.org/10.1002/stvr.430

[3] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. The dstar method for effective software fault localization. IEEE Transactions on Reliability 63, 1 (March 2014), 290-308. https://doi.org/10.1109/TR.2013.2285319

[4] Higor A. de Souza, Marcos L. Chaim, and Fabio Kon. 2016. Spectrum-Based Software Fault Localization: A Survey of Techniques, Advances, and Challenges. CoRR abs/1607.04347 (2016), 1-46. https://arxiv.org/abs/1607.04347v2

[5] Sebastian Elbaum, Gregg Rothermel, and John Penix. 2014. Techniques for improving regression testing in continuous integration development environments. In Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. ACM, New York, 235-245. https://doi.org/10.1145/2635868.2635910

[6] Bo Jiang, Zhenyu Zhang, W.K. Chan, T.H. Tse, and Tsong Yueh Chen. 2012. How well does test case prioritization integrate with statistical fault localization?. Information and Software Technology 54, 7 (July 2012), 739-758. https://doi.org/10.1016/j.infsof.2012.01.006

[7] Yanbing Yu, James A. Jones, and Mary J. Harrold. 2008. An empirical study of the effects of test-suite reduction on fault localization. In Proceedings of the 30th International Conference on Software Engineering. IEEE, Los Alamitos, CA, 201-210. https://doi.org/10.1145/1368088.1368116

[8] August Shi, Tifany Yung, Alex Gyori, and Darko Marinov. 2015. Comparing and combining test-suite reduction and regression test selection. In Proceedings of the 10th Joint Meeting on Foundations of Software Engineering. ACM, New York, 237-247. https://doi.org/10.1145/2786805.2786878

[9] Chu-Ti. Lin, Kai-Wei Tang, and Gregory M. Kapfhammer. 2014. Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests. Information and Software Technology 56, 10 (October 2014), 1322-1344. https://doi.org/10.1016/j.infsof.2014.04.013

[10] Jun-Wei Lin and Chin-Yu Huang. 2009. Analysis of test suite reduction with enhanced tie-breaking techniques. Information and Software Technology 51, 4 (April 2009), 679-690. https://doi.org/10.1016/j.infsof.2008.11.004

[11] Mary J. Harrold, Rajiv Gupta, and Mary L. Soffa. 1993. A methodology for controlling the size of a test suite. ACM Transactions on Software Engineering and Methodology 2, 3 (July 1993), 270-285. https://doi.org/10.1145/152388.152391

[12] T.Y. Chen and M.F. Lau. 1998. A new heuristic for test suite reduction, Information and Software Technology 40, 5-6 (July 1998), 347-354. https://doi.org/10.1016/S0950-5849(98)00050-0

[13] Adam M. Smith and Gregory M. Kapfhammer. 2009. An empirical study of incorporating cost into test suite reduction and prioritization. In Proceedings of the Symposium on Applied Computing. ACM, New York, 461-467. https://doi.org/10.1145/1529282.1529382

[14] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A survey on software fault localization. IEEE Transactions on Software Engineering 42, 8 (August 2016), 707-740. https://doi.org/10.1002/stvr.430

[15] W. Eric Wong, Vidroha Debroy, and Byoungju Choi. 2010. A family of code coverage-based heuristics for effective fault localization. Journal of Systems and Software 83, 2 (February 2010), 188-208. https://doi.org/10.1016/j.jss.2009.09.037

[16] Rui Abreu, Peter Zoeteweij, Rob Golsteijn, and Arjan J.C. van Gemund. 2009. A practical evaluation of spectrum-based fault localization. Journal of Systems and Software 82, 11 (November 2009), 1780-1792. https://doi.org/10.1016/j.jss.2009.06.035

[17] Ben Liblit, Mayur Naik, Alice X. Zheng, Alex Aiken, and Michael I. Jordan. 2005. Scalable statistical bug isolation. ACM SIGPLAN Notices 40, 6 (June 2005) 15-26. https://doi.org/10.1145/1064978.1065014

[18] James A. Jones and Mary J. Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering. ACM, New York, 273-282. https://doi.org/10.1145/1101908.1101949

[19] Hyunsook Do, Sebastian Elbaum, and Gregg Rothermel. 2005. Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact. Empirical Software Engineering 10, 4 (October 2005), 405-435. https://link.springer.com/article/10.1007/s10664-005-3861-2

[20] Free Software Foundation. 1988. gcov-a Test Coverage Program. Retrieved February 8, 2021 from https://gcc.gnu.org/onlinedocs/gcc/Gcov.html